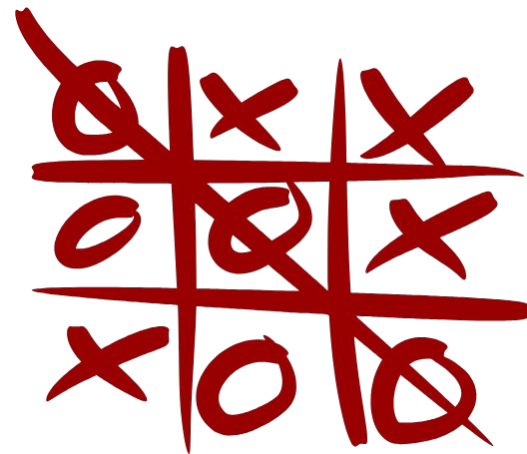




תכנות תחרותי

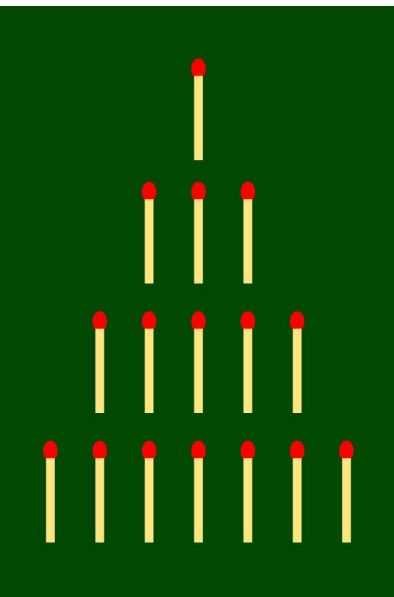
הרצאה 7



שאלה שכולנו מכירים מהרצאה ראשונה:

יש לנו ערמה עם n גפרורים ושני שחקנים, בכל תור מותר לקחת גפרור 1 או 2. מי שלוקח אחרון מנצח - כלומר: מי שלא נשאר לו מהלכים לעשות מפסיד.

פתרון אתם אמורים להכיר.



פתרון:

נבנה טבלת DP, נשים L ב-0. מכיוון שאם יש גפרור אחד נוכל להגיע ל-0 במהלך אחד לכן נשים בטבלה W, כנל לגבי 2 - וכך הלאה...

L, W, W, L, W, W, L, ...

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50

(באופן דומה לטבלה שבנינו
בשיעור הראשון - רק עכשיו
אפשר להוציא 1,2 ולא 1,2,6)

נוסחאת DP כללית למעבר:

$$win(v) = \exists \text{ move } v \rightarrow u \text{ with } win(u) = false$$

או במילים:

מצב מפסיד = כל המעברים מובילים למצב מנצח

מצב מנצח = קיים מהלך למצב מפסיד

כל מה שנעשה היום מתחיל מה-dp הזה!

הגדרה: Grundy numbers

נגדיר את הפונקציה הבאה עבור מצב v :

$$g(v) = \text{mex}\{g(u): v \rightarrow u\}$$

mex - Minimum Excluded value

פונקציה שמוגדרת על קבוצה A , ומחזירה את המספר האי-שלילי הקטן ביותר שלא שייך לקבוצה.

לדוגמא:

$$\text{mex}\{0,2,3\} = 1$$

נבנה טבלת ערכי Grundy עבור Nim:

$g(0)$	
$g(1)$	
$g(2)$	
$g(3)$	
$g(4)$	
$g(5)$	
$g(6)$	

נבנה טבלת ערכי Grundy עבור Nim:

$g(0)$	0
$g(1)$	
$g(2)$	
$g(3)$	
$g(4)$	
$g(5)$	
$g(6)$	

$$g(0) = \text{mex}\{\} = 0$$

נבנה טבלת ערכי Grundy עבור Nim:



g(0)	0
g(1)	1
g(2)	
g(3)	
g(4)	
g(5)	
g(6)	

$$g(1) = \text{mex}\{g(0)\} = \text{mex}\{0\} = 1$$

נבנה טבלת ערכי Grundy עבור Nim:



g(0)	0
g(1)	1
g(2)	2
g(3)	
g(4)	
g(5)	
g(6)	

$$g(2) = \text{mex}\{g(0), g(1)\} = \text{mex}\{0, 1\} = 2$$

נבנה טבלת ערכי Grundy עבור Nim:

g(0)	0
g(1)	1
g(2)	2
g(3)	0
g(4)	
g(5)	
g(6)	



$$g(3) = \text{mex}\{g(1), g(2)\} = \text{mex}\{1, 2\} = 0$$

נבנה טבלת ערכי Grundy עבור Nim:

g(0)	0
g(1)	1
g(2)	2
g(3)	0
g(4)	1
g(5)	
g(6)	



$$g(4) = \text{mex}\{g(2), g(3)\} = \text{mex}\{2, 0\} = 1$$

נבנה טבלת ערכי Grundy עבור Nim:

$g(0)$	0
$g(1)$	1
$g(2)$	2
$g(3)$	0
$g(4)$	1
$g(5)$	2
$g(6)$	



$$g(5) = \text{mex}\{g(3), g(4)\} = \text{mex}\{0, 1\} = 2$$

נבנה טבלת ערכי Grundy עבור Nim:

$g(0)$	0
$g(1)$	1
$g(2)$	2
$g(3)$	0
$g(4)$	1
$g(5)$	2
$g(6)$	0



$$g(6) = \text{mex}\{g(4), g(5)\} = \text{mex}\{1, 2\} = 0$$

**לא צריך להיות
גאון בשביל לראות
שיש פה חוקיות**

עכשיו נעצור לרגע ונבין משהו

מה זה אומר אם המשוואה הבאה מתקיימת:

$$g(v) = k$$

אם זה מתקיים זה אומר:

- יש מהלך למצב עם Grundy = 0
- יש מהלך למצב עם Grundy = 1
- ...
- יש מהלך למצב עם Grundy = k-1
- **אין** מהלך למצב עם Grundy = k

רואים איזשהו דמיון בין 2 הטבלאות?

Win/Lose:

0	1	2	3
4	5	6	7

Grundy values:

$g(0)$	0
$g(1)$	1
$g(2)$	2
$g(3)$	0
$g(4)$	1
$g(5)$	2
$g(6)$	0

טענה: $g(v) = 0 \iff v \text{ is losing}$

תחילה נסתכל על הייצוג של הבעיה בתור גרף - אתה מצביע לכל המצבים שאתה יכול לעבור אליהם. כלומר כל צומת v תצביע ל $v-2$ וגם ל $v-1$ (אם אפשרי). נוכיח באינדוקציה:

בסיס: מצב v , שאין ממנו שום מהלך - 0 גפרורים נותרו. אז קבוצת הילדים ריקה:

$$g(v) = \text{mex}(\emptyset) = 0$$

ומצד שני הוא כמובן losing, כי השחקן שבתור לא יכול לבצע מהלך.

המשך הוכחה:

נניח שכבר הוכחנו עבור כל המצבים שאליהם אפשר להגיע מ- v : $g(u) = 0 \Leftrightarrow u \text{ is losing}$

1) $g(v) = 0$:

מהגדרת mex מתקיים $0 \notin \{g(u) : v \rightarrow u\}$ כלומר לכל ילד u מתקיים $g(u) \neq 0$

לפי הנחת האינדוקציה, כל הילדים הם winning. לכן כל מהלך שאני עושה מעביר את היריב למצב winning. אזי v losing.

2) $g(v) > 0$:

כדי שה-mex יהיה חיובי, 0 חייב להופיע בקבוצה. לכן:

$\exists u \text{ child of } v : g(u) = 0 \Rightarrow u \text{ is losing} \Rightarrow v \rightarrow u \text{ is winning} \Rightarrow v \text{ is winning}$

לכן: $g(v) = 0 \Leftrightarrow v \text{ is losing}$

טיפה שיקרנו...

ה - NIM שהראנו לכם הוא NIM מזויף. והוא הוא רק דומה ל - NIM האמיתי.

ה-NIM האמיתי:

- נתונת k ערמות ובכל ערימה יש מספר גפרורים a_i
- בכל תור בוחרים ערימה אחת ומקטינים אותה לכל ערך קטן יותר ממש.
- המפסיד הוא מי שבתורו כל הערמות ריקות.

שאלה: עבור ערימה i מה ה - Grundy value שלה?

עבור ערימה אחת:

מערימה בגודל n ניתן לעבור לערכים:

$0, 1, 2, \dots, n-1$

לכן:

$$g(n) = \text{mex}\{0, 1, 2, \dots, n-1\} = n$$

כלומר לערימה מגודל n מתקיים: Grundy value = n

רגע נעצור ונחשוב איך פותרים את NIM?

NIM:

- נתונת k ערמות ובכל ערימה יש מספר גפרורים a_i
- בכל תור בוחרים ערימה אחת ומקטינים אותה לכל ערך קטן יותר ממש.
- המפסיד הוא מי שבתורו כל הערמות ריקות.

נגלה לכם שהאסטרטגיה עבור NIM ב-2 ערימות הוא כל פעם לדאוג לכך שבסוף המהלך שלך, גודל 2 הערמות יהיה זהה - ואם הוא מתחיל זהה, הפסדת.

אבל ב-3 ערימות זה מתחיל להסתבך.

למי יש רעיון לפתרון?

פתרון נאיבי - dp לכל המצבים

עבור nim עם לדוגמא 3 ערימות a, b, c - נאחד את שלושת המצבים לאחד (אוטומט מכפלה):

$$g(a, b, c) = \text{mex}\{g(0, b, c), g(1, b, c), \dots, (a, b, c-1)\}$$

ובאופן דומה לפתרון עבור ערימה אחת: $g(v) = 0 \Leftrightarrow v \text{ is losing}$

סיבוכיות:

אם כל ערימה היא בערך $1e9$ אז מספר החישובים מתפוצץ ויוצא $1e27$. לא טוב בכלל:

עכשיו השאלה: האם באמת צריך לחשב Grundy לכל השלשה יחד?

Sprague-Grundy theorem

אבל רגע לפני - הגדרה:

לפני שנספר לכם מה הלמה המרכזית של השיעור אומרת, צריך הגדרה.

משחק שיוויוני ונורמלי (normal + impartial game בלעז):

משחק הוא impartial+normal אם הוא משחק עם מספר מהלכים סופי, למי שנגמרו המהלכים - מפסיד, וגם מכל state, לשני השחקנים יש בדיוק את אותה קבוצת מהלכים חוקיים.

כלומר חוקיות המהלך תלויה רק ב-state, ולא בזהות השחקן. למשל Nim הוא impartial:

אם המצב הוא (3,5,7) אז לא משנה אם זה תור של Alice או Bob - לשניהם מותר לבחור ערימה אחת ולהקטין אותה.

לעומת זאת שחמט הוא לא impartial, כי לשחקן אחד מותר להזיז את הכלים הלבנים ולשני את השחורים, כלומר קבוצת המהלכים תלויה בשחקן.

Sprague-Grundy theorem

משפט Sprague-Grundy אומר שכל מצב במשחק impartial שקול לערימת Nim אחת, שגודלה הוא ערך ה-Grundy של המצב.

זו מסקנה מטורפת בפני עצמה, אבל מעבר לכך הוא גם אומר טענה נוספת:

אם יש כמה משחקים שוויונים בלתי תלויים $G_1 + G_2 + \dots + G_k$ ובכל תור בוחרים **משחק אחד** ועושים בו מהלך (כמו שבוחרים ערימת ב-Nim ועושים בה מהלך), אז מתקיים:

$$g(G_1 + \dots + G_k) = g(G_1) \oplus \dots \oplus g(G_k)$$

מקרה פרטי: למה XOR עובד ב-Nim?

נתונת k ערמות ובכל ערימה יש מספר גפרורים a_i ובכל תור בוחרים ערמה ומקטינים אותה.
ברור פה שהמשחקים הם impartial ובלתי תלויים לכן הם צריכים לקיים את הלמה.

$$\text{נסמן: } X = a_1 \oplus \dots \oplus a_n$$

נרצה להוכיח: $X = 0 \Leftrightarrow \text{we are in a losing state}$

טענה א': אם $X=0$, כל מהלך הופך אותו ל-nonzero

נניח ששינינו רק את הערימה ה- i .

$$a_i \rightarrow a'_i \quad a'_i < a_i$$

$$X' = \underbrace{X}_{0} \oplus a_i \oplus a'_i = a_i \oplus a'_i$$

$$a_i \neq a'_i \Rightarrow a_i \oplus a'_i \neq 0 \Rightarrow X' \neq 0$$

ולכן אם $X=0$, כל מהלך הופך אותו ל-nonzero

טענה ב': אם X שונה מ-0 תמיד ניתן להגיע ל- $X=0$

$$a'_i < a_i$$

$$X' = X \oplus a_i \oplus a'_i = 0 \Rightarrow a'_i = X \oplus a_i$$

למה זה מהלך חוקי?

ניקח את הביט הגבוה ביותר הדולק ב- X . מכיוון שהביט הזה ב-XOR הכולל הוא 1, חייבת להיות לפחות ערימה אחת i שבה גם הביט הזה הוא 1. לכן ב-xor עם הערימה הזו, הביט הזה מתבטל ונוכל להפוך את הערימה לערך a' מכיוון שהוא קטן מ- a .

מסקנה

המצב הסופי $(0, \dots, 0, 0)$ הוא losing ויש לו $XOR = 0$.
אם סכום ה-XOR של כל הערימות הוא 0, אז לא משנה איזה מהלך עושים - ה-XOR יהפוך להיות שונה מ-0.

לעומת זאת, אם סכום ה-XOR שונה מ-0, תמיד קיימת ערימה שאפשר להקטין כך שאחרי המהלך ה-XOR יהיה בדיוק 0.

לכן מצב עם XOR שווה ל-0 הוא מצב מפסיד: אין ממנו שום מהלך למצב מפסיד אחר.

ומצב עם XOR שונה מ-0 הוא מצב מנצח: יש ממנו מהלך שמעביר את היריב למצב עם XOR שווה ל-0, כלומר למצב מפסיד.

כלומר:

$X = 0 \iff \text{we are in a losing state}$

פתרון NIM:

קל ופשוט: לחשב את X שהוא xor של כל האיברים.

אם $X=0$ תחזיר השחקן השני.

אחרת תחזיר שהשחקן הראשון מנצח.

עכשיו בחזרה ל-Sprague-Grundy

נניח שיש משחק impartial כללי G .

נגדיר: $g(G) = \text{mex}\{g(G') : G \rightarrow G'\}$

אז כמו שאמרנו מקודם מבחינת Grundy מתקיים:

$$g(G) = k \Rightarrow G \rightarrow k-1, k-2, \dots, 0$$

בדיוק כמו ערימת Nim מגודל k :

$$k \rightarrow k-1, k-2, \dots, 0$$

והמשפט אומר:

$$G \equiv \text{Nim heap of size } g(G)$$

לא אומרים שהגרפים זהים. יכול להיות של- G יש גם מהלכים למצבים עם Grundy 17 או 100.

אבל מבחינת ההתנהגות הרלוונטית למשחק, mex גורם לו להתנהג כמו ערימת Nim בגודל $g(G)$

שאלה:

נתון DAG, ועל כל צומת יש מספר אי שלילי של אסימונים.
המשחק מוגדר באופן הבא: בכל תור בוחרים אסימון ומזיזים אותו לאורך קשת אחת. מי שאין לו מהלך מפסיד.

תיקחו כמה דקות לחשוב על כיוון.

אבחנה: פירוק למשחקים

רעיון אחד הוא לפרק את המשחקים לפי הצמתים, אבל מהר מאוד רואים שהמשחקים הם לא בלתי תלויים - כאשר במהלך אחד מעבירים בין צומת A ל-B, שני הסכומים שלהם משתנים.

אבחנה: פירוק למשחקים

רעיון אחד הוא לפרק את המשחקים לפי הצמתים, אבל מהר מאוד רואים שהמשחקים הם לא בלתי תלויים - כאשר במהלך אחד מעבירים בין צומת A ל-B, שני הסכומים שלהם משתנים.

במקום: נפרק לאסימונים!

כל אסימון נתייחס אליו בתור משחק משל עצמו, עם חוקים זהים למשחק המקורי. ברור כי המשחקים הם בלתי תלויים, נורמלים ושיוויניים לכן נוכל להשתמש במשפט שלמדנו.

פתרון

עבור אסימון אחד בצומת v נסתכל על פונקצית ה-Grundy שלו:

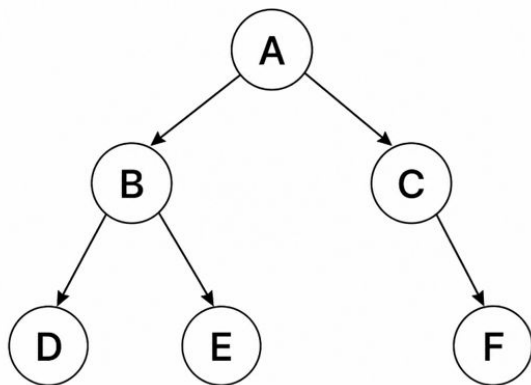
$$g(v) = \text{mex}\{g(u) : v \rightarrow u\} \Rightarrow g(\text{sink}) = 0$$

$$g(D) = g(E) = g(F) = 0$$

$$g(B) = \text{mex}\{0, 0\} = 1$$

$$g(C) = \text{mex}\{0\} = 1$$

$$g(A) = \text{mex}\{1, 1\} = 0$$



מכיוון שכל אסימון הוא משחק עצמאי, נגיד שאסימונים ממוקמים בצמתים $a_1, a_2, \dots, a_n$ על פי המשפט שלמדנו:

$$g_{total} = g(a_1) \oplus g(a_2) \oplus \dots \oplus g(a_n)$$

כלומר:

$$\text{token at vertex } v \equiv \text{Nim pile of size } g(v)$$

לכן נחזיר שהשחקן הראשון מנצח אם $g_{total} \neq 0$ ואחרת שהשחקן השני.

איך מחשבים פה mex מהר?

אם ל- v יש d ילדים, אז $g(v) \leq d$.

לכן מספיק לסמן את ערכי ה-Grundy האמיתיים של הילדים שנמצאים בטווח $0, \dots, d$. כל ערך גדול מ- d אפשר להתעלם ממנו, כי הוא לא יכול להשפיע על ה-mex.

לדוגמא:

אם יש לי 3 ילדים ואת הערכים 0, 2, 100 - אז אפשר "להתעלם" מ-100 ולמלא מערך של מספרים עד גודל d ובמערך הזה אנחנו רואים ש-1 חסר.

לפני השאלה הבאה - bitmask

כשיש לנו קבוצה קטנה של אפשרויות, אפשר לשמור אותה בתוך מספר אחד.
כל bit מייצג האם אפשרות מסוימת פעילה. לדוגמה, אם המהלכים האפשריים הם:
1,2,3,4 אפשר לייצג את המצב:
1101 - שהוא אומר, מותר לך לעשות את כל המהלכים חוץ מ-2.
bitmask היא דרך חכמה לשמור subset בתוך מספר אחד.

TIP

```
int mask = 0;
// add 2
mask |= (1 << 2);
// check if 2 exists
if (mask & (1 << 2))
    cout << "yes\n";
// remove 2
mask &= ~(1 << 2);
// toggle 3
mask ^= (1 << 3);

bitset<8> bitmask;
bitmask.set(position: 2);           // add 2
bitmask.reset(2);                   // remove 2
bitmask.flip(3);                    // toggle 3
if (bitmask[2])
    cout << "exists";
```

שאלה

Sam has been teaching Jon the *Game of Stones* to sharpen his mind and help him devise a strategy to fight the white walkers. The rules of this game are quite simple:

- The game starts with n piles of stones indexed from 1 to n . The i -th pile contains s_i stones.
- The players make their moves alternatively. A move is considered as removal of some number of stones from a pile. Removal of 0 stones does not count as a move.
- The player who is unable to make a move loses.

Now Jon believes that he is ready for battle, but Sam does not think so. To prove his argument, Sam suggested that they play a modified version of the game.

In this modified version, no move can be made more than once on a pile. For example, if 4 stones are removed from a pile, 4 stones cannot be removed from that pile again.

Sam sets up the game and makes the first move. Jon believes that Sam is just trying to prevent him from going to battle. Jon wants to know if he can win if both play optimally.

First line consists of a single integer n ($1 \leq n \leq 10^6$) — the number of piles.

Each of next n lines contains an integer s_i ($1 \leq s_i \leq 60$) — the number of stones in i -th pile.

מה ה-state

גודל הערימה לבד לא מספיק.

נגדיר:

$$\text{state} = (s, \text{mask})$$

כאשר s זה מספר האבנים שנשארו,
והביט ה- k ב- mask לא דולק (שווה ל-0) אם השתמשנו במהלך של k אבנים.

מצב התחלתי:

$$(s, 11 \dots 11)$$

המעברים

נניח שאנחנו במצב:

$$\text{state} = (s, \text{mask})$$

ורוצים להוריד k אבנים, המהלך חוקי אם k גדול שווה ל- s וגם הביט ה- k דולק.

המעבר יראה באופן הבא:

$$(s, \text{mask}) \rightarrow (s - k, \text{mask} \setminus \{k\})$$

Grundy

$$g(s, mask) = mex\{for\ every\ legal\ k\ |\ g(s - k, mask \setminus \{k\})\}$$

האם פתרנו?

לא בדיוק - יש 60 מהלכים שונים, לכן mask יכול לקבל ערכים עד 2^{60}

האם באמת כל הערכים של mask יכולים להופיע?

אילו Masks בכלל אפשריים

נניח שהורדנו מספר מסויים של אבנים.

הערימה התחילה עם עד 60 אבנים, לכן:

$$\sum_{k \in \text{mask}} k \leq 60$$

נשים לב שגם במקרה הכי חסכוני שלוקחים את הביטים הקטנים ביותר, יכולים להיות עד 10 ביטים שהשתמשנו בהם בכל פעם.

כלומר אנחנו לא צריכים לדעת כמה states יש בדיוק - מספיק לדעת שה-DFS יוצר רק states חוקיים, ורוב מרחב ה-bitmask לעולם לא נבקר בו.

למי שרוצה לדעת כמה masks חוקיים יש

$q(s)$ = number of combinations to create s from different numbers

$$\#masks = \sum_0^{60} q(s, 60) = 101983$$

$$q(s, x) = \underbrace{q(s, x-1) + q(s-x, x-1)}_{\text{you can write dp to get the answer}}$$

אפשר לצמצם אפילו יותר!

נניח שנשארו בערימה רק x אבנים, ולא השתמשנו במהלך שמוריד $x+1$. האם זה משנה לנו? לא.

אז כל מידע ב $mask$ מעבר לערכים גדולים מ- s כבר לא רלוונטי. אפשר למחוק:

$$mask \leftarrow mask \& ((1LL \ll s) - 1)$$

אז עכשיו אפשר פשוט לחשב את כל המצבים התקינים ולשמור אותם. וסיימנו!

קוד

```
map<pii,int> dp;
int grundy(int s, int mask) {
    mask &= (1LL << s) - 1;
    pii state = {x: [&s, y: [&mask]};
    if (dp.count(state))
        return dp[state];
    bool seen[61] = {};
    rep(k,0,s) {
        if (!(mask & (1LL << k)))
            continue;

        int nxt = grundy(
            s: s - k - 1,
            mask ^ (1LL << k)
        );

        seen[nxt] = true;
    }
    int mex = 0;
    while (seen[mex])
        mex++;
    return dp[state] = mex;
}
```

חשוב ש-int יהיה long long,
צריך לייצג לפחות 60 ביטים

```
void solve() {
    vi g(n:61);
    rep(s,1,61)
        g[s] = grundy(s, mask: (1LL << s) - 1);
    int n;
    cin >> n;
    int xr = 0;
    rep(i,0,n) {
        int s;
        cin >> s;
        xr ^= g[s];
    }
    cout << (xr == 0 ? "YES" : "NO") << '\n';
}
```

הסוף

דוגמא נוספת: משחק המדרגות

יש מטבעות על אוסף של מדרגות מ-1 עד n . על כל מדרגה מספר המטבעות שבה הוא $a[i]$.
שני שחקנים משחקים בתורות. בכל תור, השחקן בוחר מדרגה i שיש עליה לפחות מטבע אחד,
ובוחר מספר חיובי של מטבעות c . הוא מעביר את c המטבעות האלה מהמדרגה i למדרגה $i-1$.
מדרגה 0 נחשבת מחוץ למשחק, ולכן מטבע שמגיע אליה נעלם.
השחקן שלא יכול לבצע מהלך מפסיד.
בהינתן המערך a , מי מנצח?

פתרון משחק המדרגות:

נסתכל על: $X = a_1 \oplus a_3 \oplus a_5 \oplus \dots$ כלומר נתעלם כרגע מהמדרגות הזוגיות.

אבחנה: בכל מהלך משתנה בדיוק ערך אחד מבין $a_1, a_3, \dots$

אם מעבירים מטבעות מאי זוגי לזוגי אותו a_i קטן, ואם מעבירים מזוגי לאי זוגי אותו a_i גדל.

לכן אם לפני המהלך $X = 0$ לאחר המהלך:

$$X' = X \oplus a_i \oplus a'_i = a_i \oplus a'_i \neq 0$$

כלומר אם $X = 0$ אי אפשר לעבור ל- $X = 0$

מצד שני אם $X \neq 0$ נוכל לייצר a'_i כמו שייצרנו ב-Nim בשביל לאפס את X . כלומר קיים מהלך ל- $X=0$.

לכן:

$$X = 0 \Leftrightarrow \text{losing OR } X \neq 0 \Leftrightarrow \text{winning}$$